

Mediso

Smart Hospital Management System

Project Report

Team Member	Role
Satyam Swarnakar	System Architect
Avneet Singh	Project Lead
Kanishka Dubey	Full Stack Developer
Tanisha	Frontend and Admin Backend
Manpreet Singh	Backend Developer

April 2026

1. Problem Statement and Solution Approach

Healthcare management in most hospitals and clinics continues to rely on manual, paper-based processes for scheduling, patient records, and prescription management. These systems suffer from inefficiency, data loss, difficulty in coordination between departments, and poor patient experience. Patients often wait long periods to book appointments, have no visibility into their medical history, and struggle to communicate effectively with doctors.

The goal of Mediso is to digitize and streamline these operations through a centralized, role-based web platform. The system serves three distinct actors: patients who need to book appointments and access their medical history, doctors who need to manage their schedules and write prescriptions, and admins who need to oversee the platform, manage users, and monitor system health.

The solution is built as a full-stack web application using Next.js for the frontend and Express.js with PostgreSQL for the backend. Authentication is handled through JWT tokens, and role-based access control ensures each user sees only what is relevant to their role. The architecture is designed to be modular and extensible, allowing new features to be added without disrupting existing functionality.

2. Team Member Contributions

Member	Role	Key Contributions
Satyam Swarnakar	System Architect	Designed all four UML diagrams (Class, ER, Sequence, Use Case). Defined the system architecture, entity relationships, and data flow. Established the structural foundation that guided the entire development process.
Avneet Singh	Project Lead	Led backend deployment on Render, configured PostgreSQL environment, managed seed data, fixed production deployment errors, coordinated team integration and pull requests, and set up the monorepo folder structure.
Kanishka Dubey	Full Stack Developer	Implemented authentication (login and registration) with database-backed services using Prisma, built the doctor dashboard UI and APIs, and developed the complete patient module including dashboard, doctors, appointments, history, and profile pages.

Tanisha	Frontend and Admin Backend	Built core UI components and layout (sidebar, top header, shared cards), developed the admin panel for doctor and patient management, created the admin dashboard and appointment page UI, and contributed UI component library files.
Manpreet Singh	Backend Developer	Implemented doctor-related backend APIs, configured frontend and backend environment variables, resolved TypeScript and ESLint errors blocking Vercel builds, integrated the admin doctors module, and set up ignore files and project configuration.

3. System Design and Architecture

3.1 Three-Tier Architecture

Mediso is structured as a three-tier application. The presentation tier is a Next.js 14 application using the App Router with role-based route groups. The application tier is a Node.js and Express.js REST API that handles business logic, authentication, and data transformation. The data tier uses PostgreSQL accessed through Prisma ORM, which handles schema migrations and type-safe queries.

3.2 Authentication and Authorization

On successful login, the backend issues a signed JWT containing the user's ID, email, role, and name. The token is stored in localStorage on the client. An Axios interceptor automatically attaches the Authorization header to every outgoing API request. On the server, middleware validates the token signature and expiry before any controller logic runs. A separate role middleware checks the decoded role against the required role for each route, returning 403 Forbidden for unauthorized access.

3.3 Role-Based Routing

The frontend organizes pages into three route groups: /admin, /doctor, and /patient. Each group has its own layout with a role-specific sidebar and navigation. An AuthGuard component on each layout reads the decoded JWT role and redirects to the login page if the token is missing or the role does not match the current route. This double-layer protection (frontend guard plus backend middleware) ensures users cannot access data outside their role.

3.4 Scalability Considerations

The service layer in the backend separates database queries from HTTP handling, making it straightforward to replace or optimize individual data access functions without touching controllers.

Prisma's connection pooling handles concurrent database requests efficiently. The frontend's service modules abstract all API calls, so the base URL can be swapped between environments using a single environment variable. Deployment on Vercel (frontend) and Render (backend) allows each tier to scale independently.

4. Object-Oriented Programming Concepts

4.1 Encapsulation

The service modules in both frontend and backend encapsulate all data access logic behind clean function interfaces. For example, ``doctor.service.ts`` on the frontend groups all doctor-related API calls (availability slots, prescriptions, stats) behind exported functions. Callers do not need to know the HTTP method, URL path, or response shape. Similarly, the backend's ``auth.service.ts`` encapsulates bcrypt hashing and JWT signing so controllers only call high-level functions like ``registerUser`` or ``loginUser``.

4.2 Abstraction

The Axios instance in ``lib/axios.ts`` abstracts away the repeated concerns of setting the base URL and attaching the Authorization header. Components throughout the application import this configured instance rather than raw Axios, hiding the network configuration details. On the backend, the Prisma client is initialized once and exported from ``lib/prisma.ts``, abstracting the connection lifecycle from all service files.

4.3 Inheritance

The User entity serves as the base type in the system. Doctor, Patient, and Admin extend User by adding role-specific fields. In the database schema, Doctor, Patient, and Admin tables each contain a foreign key to the User table, implementing single-table inheritance. On the TypeScript side, types like ``DecodedToken`` extend common base interfaces to carry additional role-specific fields.

4.4 Polymorphism

The appointment status field demonstrates polymorphism in rendering. The ``AppointmentCard`` component renders different badge colors and styles depending on whether the status is pending, confirmed, completed, or cancelled. The same component handles all four cases through a status-to-style mapping object, meaning the rendering logic is driven by data rather than duplicated conditionals. Similarly, the sidebar navigation dynamically renders different menu items based on the authenticated user's role.

5. Design Patterns

5.1 Repository Pattern

All database access is separated into service and repository files. The backend's `dashboard.repository.ts` contains raw Prisma queries for dashboard statistics, keeping controllers free of data access logic. This makes it possible to swap the underlying database or ORM without modifying any HTTP handling code. Each service file acts as a repository that owns queries for a specific domain.

5.2 Singleton Pattern

The Prisma client is instantiated once in `lib/prisma.ts` and re-exported as a singleton. In production, this prevents exhausting the database connection pool. In development, Next.js hot reloading can create multiple Prisma instances, so the singleton is cached on the global object to avoid connection leaks. The Axios instance similarly follows the singleton pattern by creating one configured client that all service modules share.

5.3 Observer Pattern (Notification Flow)

The notification system follows an observer-like pattern. When an appointment is booked or updated, the backend creates a Notification record for the relevant user. The patient dashboard polls for notification counts and displays recent alerts, effectively observing changes in system state. The doctor receives a new appointment alert when a patient books a slot.

5.4 Strategy Pattern

Authentication strategy is applied through middleware composition. Different route groups use different middleware combinations: public routes use no middleware, patient routes use `authenticate` middleware, and admin routes use both `authenticate` and `requireRole('ADMIN')` middleware. Swapping or extending the authorization strategy for a route group requires only changing which middleware functions are applied.

6. SOLID Principles

S - Single Responsibility Principle

Each file in the codebase has a focused purpose. Controllers only handle HTTP request and response. Service files only contain business logic and data access. Middleware files only contain

cross-cutting concerns like authentication. UI components are split by domain, with separate files for AppointmentCard, HistoryCard, ProfileCard, and SummaryCard, each responsible for rendering a single piece of the interface.

O - Open/Closed Principle

The status-to-style mapping in AppointmentCard is open for extension. Adding a new appointment status requires only adding a new key to the mapping object, with no changes to the component logic. Similarly, new API endpoints can be added to the backend by creating a new controller file and registering a new route, without modifying existing controllers.

L - Liskov Substitution Principle

The User base type can be substituted with any of its subtypes (Doctor, Patient, Admin) without breaking the authentication flow. JWT middleware extracts a `DecodedToken` with a role field, and the rest of the system branches on that role without needing to know the concrete subtype. Any valid user token works uniformly through the authorization pipeline.

I - Interface Segregation Principle

TypeScript interfaces are kept narrow and role-specific. The `PatientProfile` interface contains only patient-relevant fields. The `DoctorStats` interface contains only doctor dashboard data. The `DashboardStats` interface is specific to admin data. No component or service is forced to depend on fields from another role's interface.

D - Dependency Inversion Principle

High-level modules like pages and dashboard components depend on service function abstractions rather than directly on Axios or Prisma. For example, the patient dashboard page imports `getDoctorAppointments` from the service layer, not from Axios directly. This means the underlying HTTP client could be replaced without changing any page component.

7. UML Diagrams

7.1 Class Diagram

The class diagram below shows the core entities and their relationships. User is the base class from which Patient, Doctor, and Admin inherit. A Patient can book zero or more Appointments. A Doctor can attend zero or more Appointments. Each Appointment generates exactly one Prescription. A User receives zero or more Notifications.

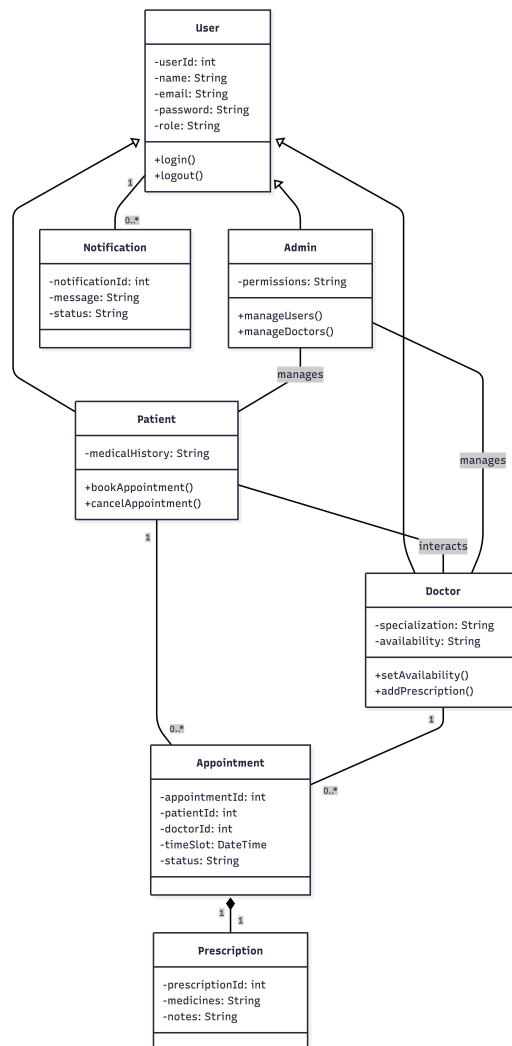


Figure 1: Class Diagram showing system entities, attributes, methods, and relationships

7.2 ER Diagram

The Entity-Relationship diagram below maps the database schema. The USER table serves as the central entity with one-to-one relationships to PATIENT, DOCTOR, and ADMIN tables via a shared user_id foreign key. APPOINTMENT connects PATIENT and DOCTOR through many-to-one relationships. PRESCRIPTION has a one-to-one relationship with APPOINTMENT. NOTIFICATION links back to USER with a many-to-one relationship.

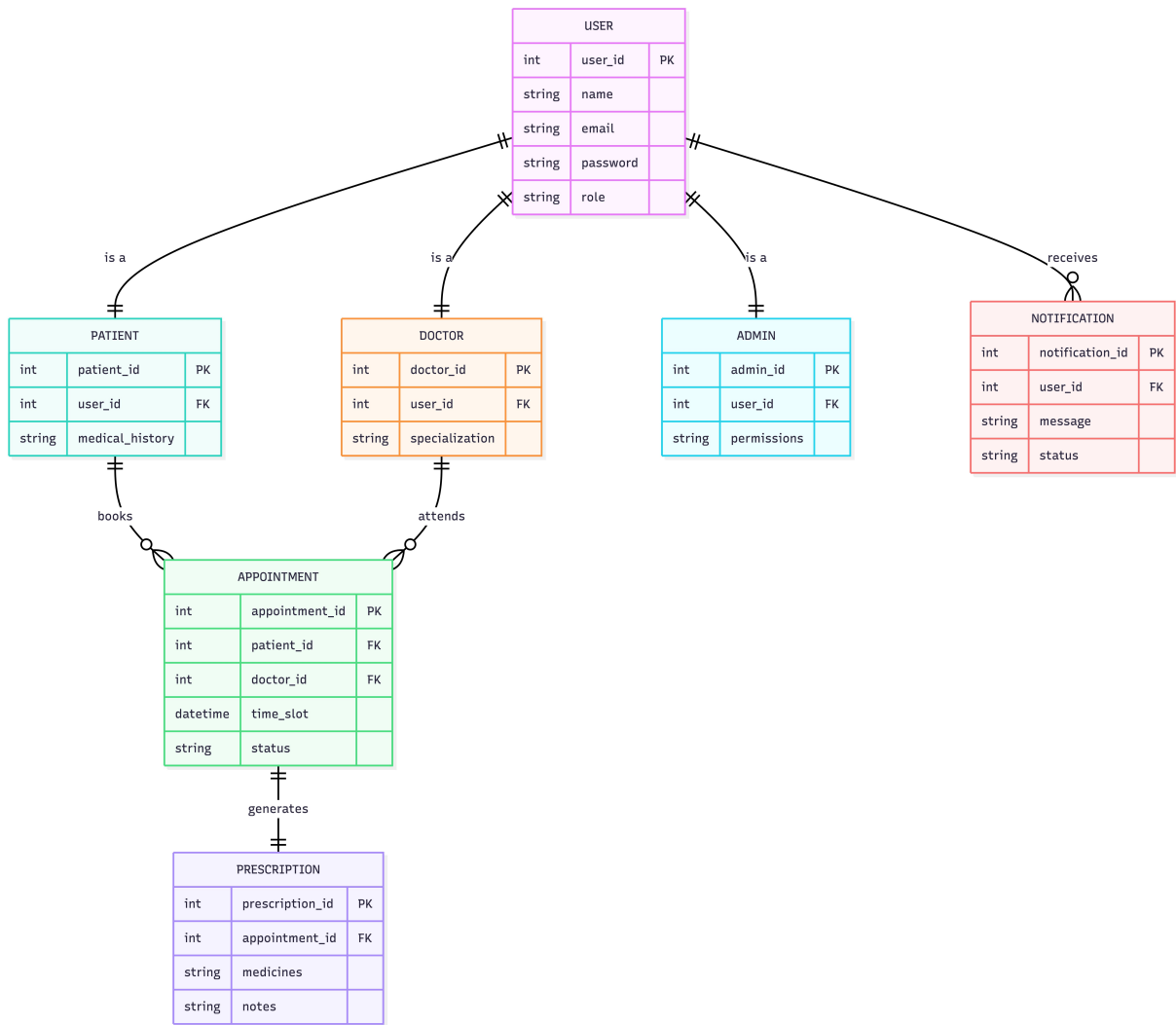


Figure 2: ER Diagram showing database schema and entity relationships

7.3 Sequence Diagram

The sequence diagram illustrates the appointment booking workflow, which is the primary use case of the system. The patient logs in through the UI, which sends credentials to the Controller API. Upon successful authentication, the patient browses available doctors, selects a doctor, and views available time slots. The Appointment Service validates slot availability against the database, creates the appointment record, triggers a notification to the doctor, and returns a confirmation to the patient.

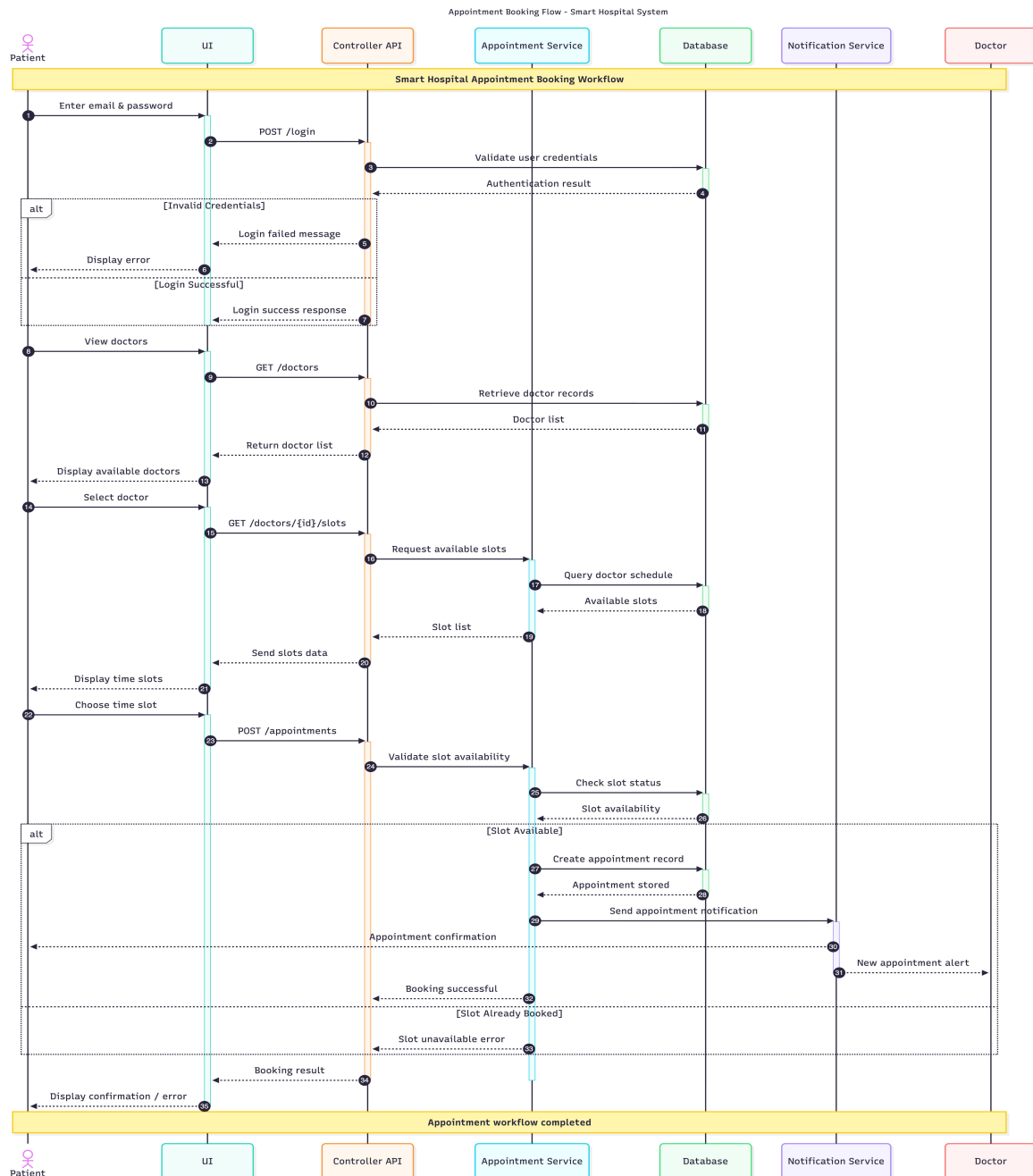


Figure 3: Sequence Diagram for the Appointment Booking workflow

7.4 Use Case Diagram

The use case diagram captures the interactions between the three actors and the system. Patients can view medical history, cancel appointments, book appointments, and view doctors. Doctors can add prescriptions, view appointments, and set their availability. All three actors (Patient, Doctor, Admin) share Login and Register use cases. Admins additionally manage doctors, manage users, and monitor the system.

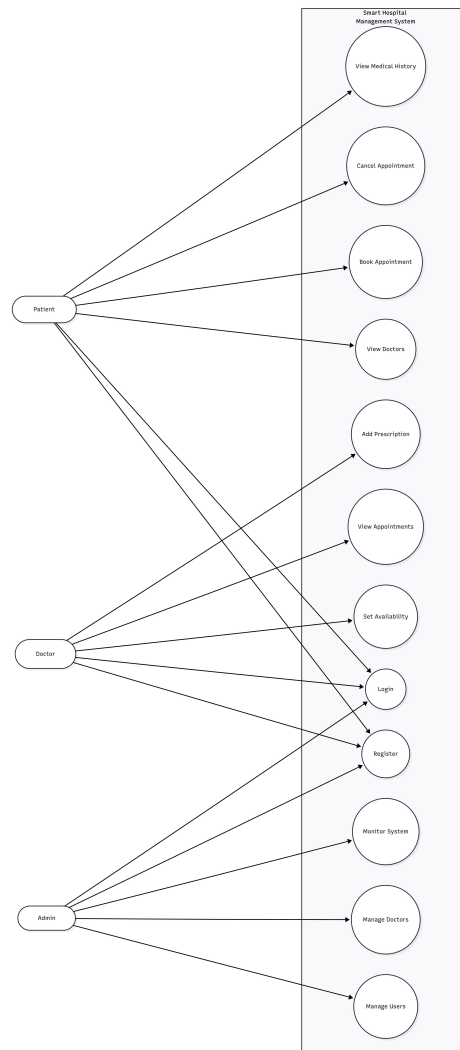


Figure 4: Use Case Diagram showing actor interactions with the system

8. Application Screenshots

8.1 Login Page

The login screen presents a clean, minimal interface with the Mediso brand. Patients, doctors, and admins all use the same login endpoint, with the server returning a role-specific JWT that routes them to the correct dashboard.

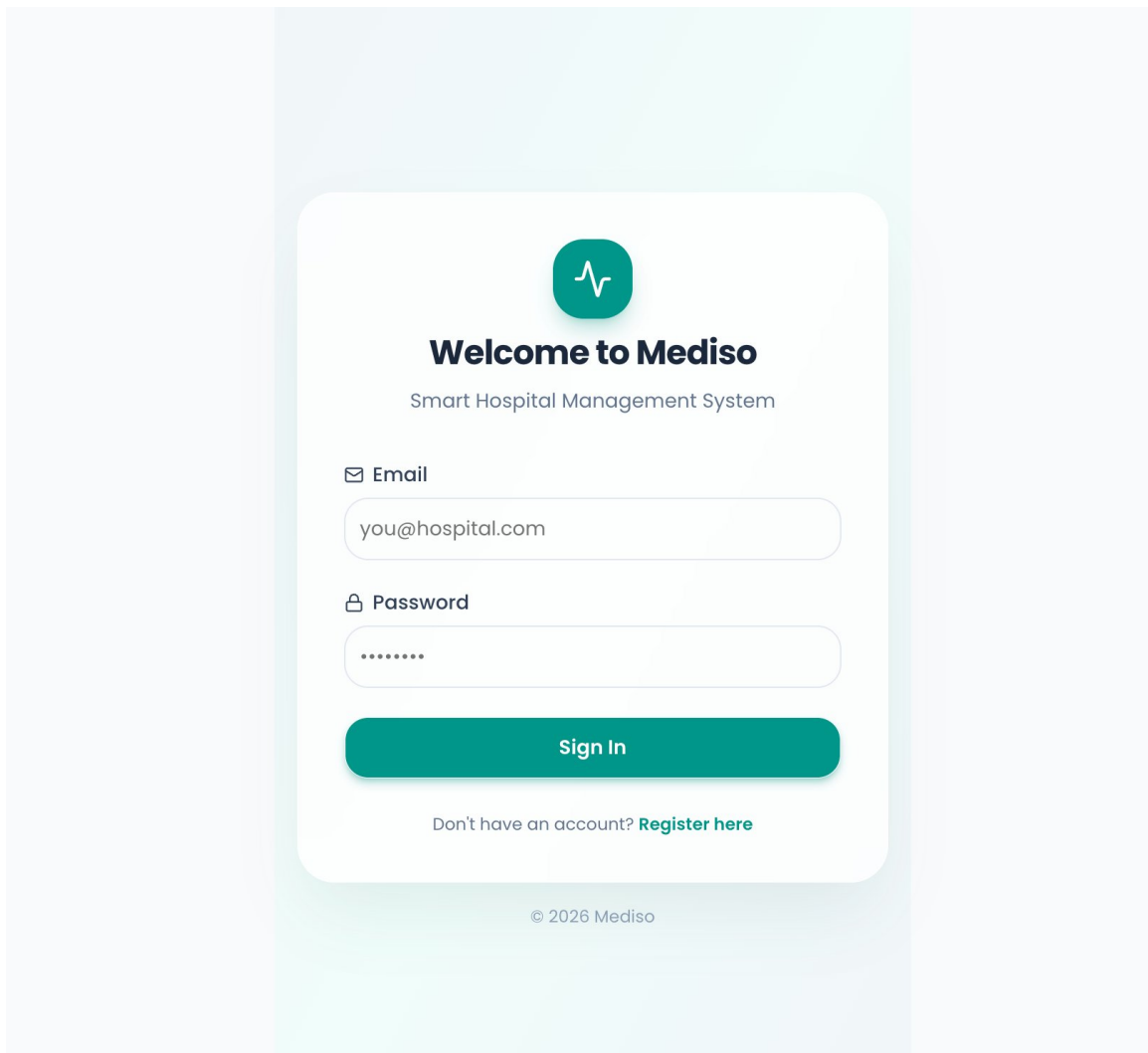


Figure 5: Mediso Login Page

8.2 Patient Dashboard

The patient dashboard provides a summary of upcoming appointments, total appointments, prescriptions, and notification count. Upcoming appointments are listed with doctor name, date,

time, and status. Quick action buttons allow the patient to book a new appointment, view doctors, or navigate to medical history.

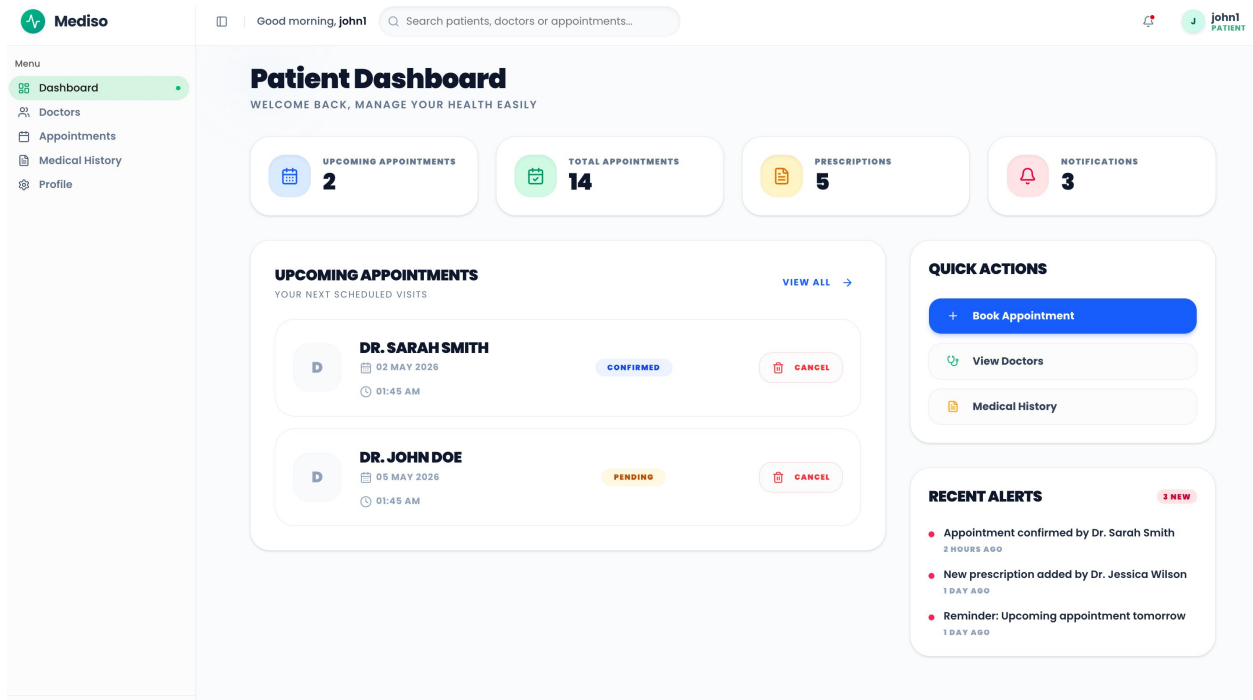


Figure 6: Patient Dashboard

8.3 Find Doctors Page

Patients can browse all available doctors, filter by specialization (Cardiologist, Dentist, Neurologist, General), and search by name. Each doctor card shows their rating and availability status. Doctors marked as not available have a greyed-out booking button.

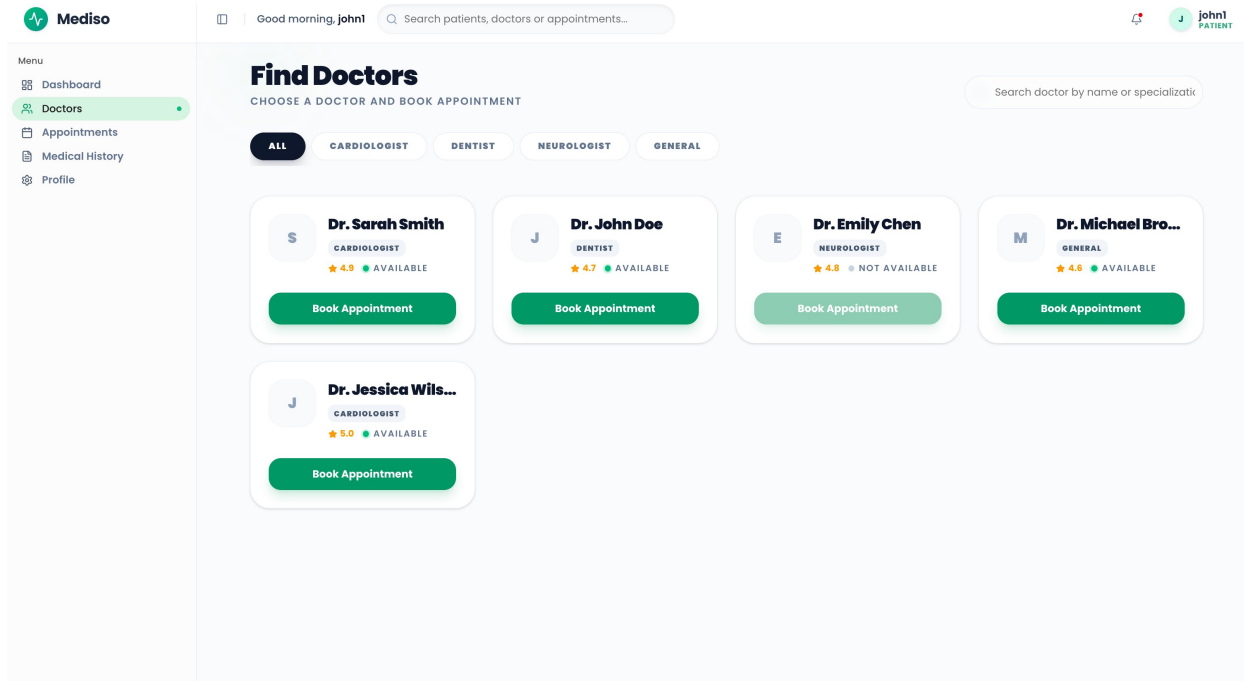


Figure 7: Find Doctors Page

8.4 My Appointments Page

The appointments page lists all of a patient's appointments with status indicators (Confirmed, Pending, Completed, Cancelled). Patients can filter by All, Upcoming, or Completed. Active appointments show a Cancel button. This page uses the AppointmentCard component with dynamic status-based styling.

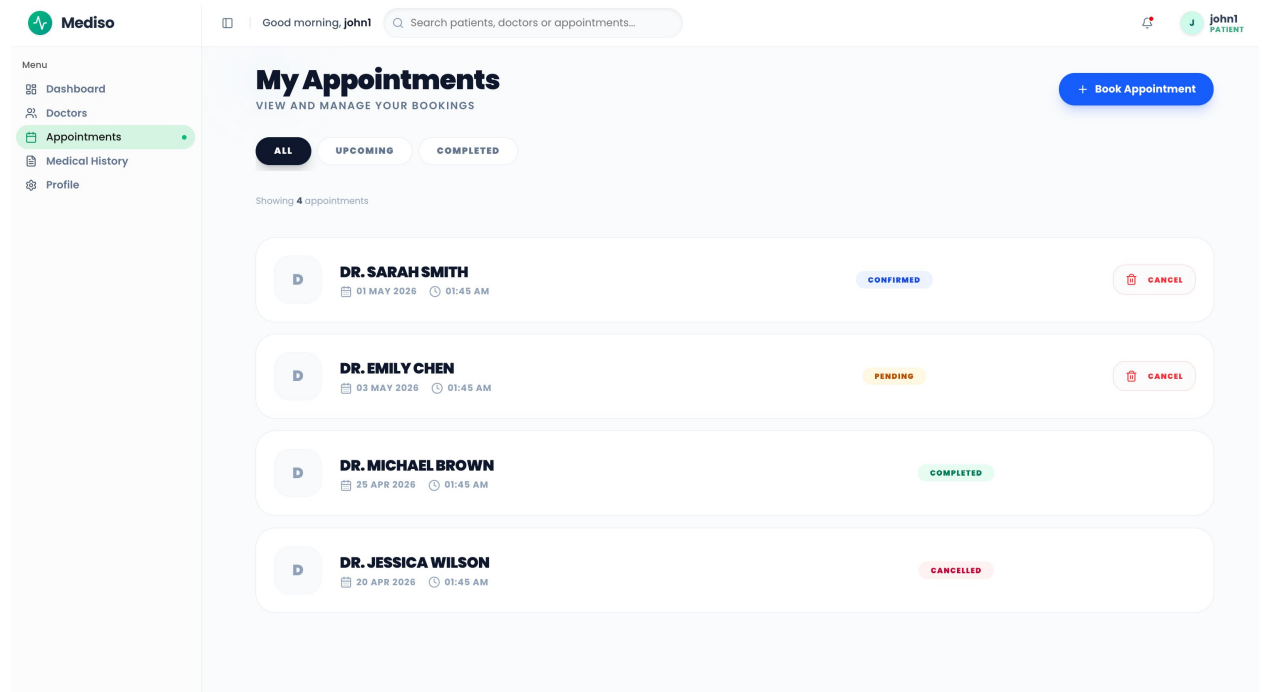


Figure 8: My Appointments Page

8.5 Medical History Page

The medical history page shows completed appointments with diagnosis and prescribed medicines. Patients can search by doctor name or date, and filter by last 7 days or last 30 days. Each history card provides a View Details button that opens a modal with the full prescription information.

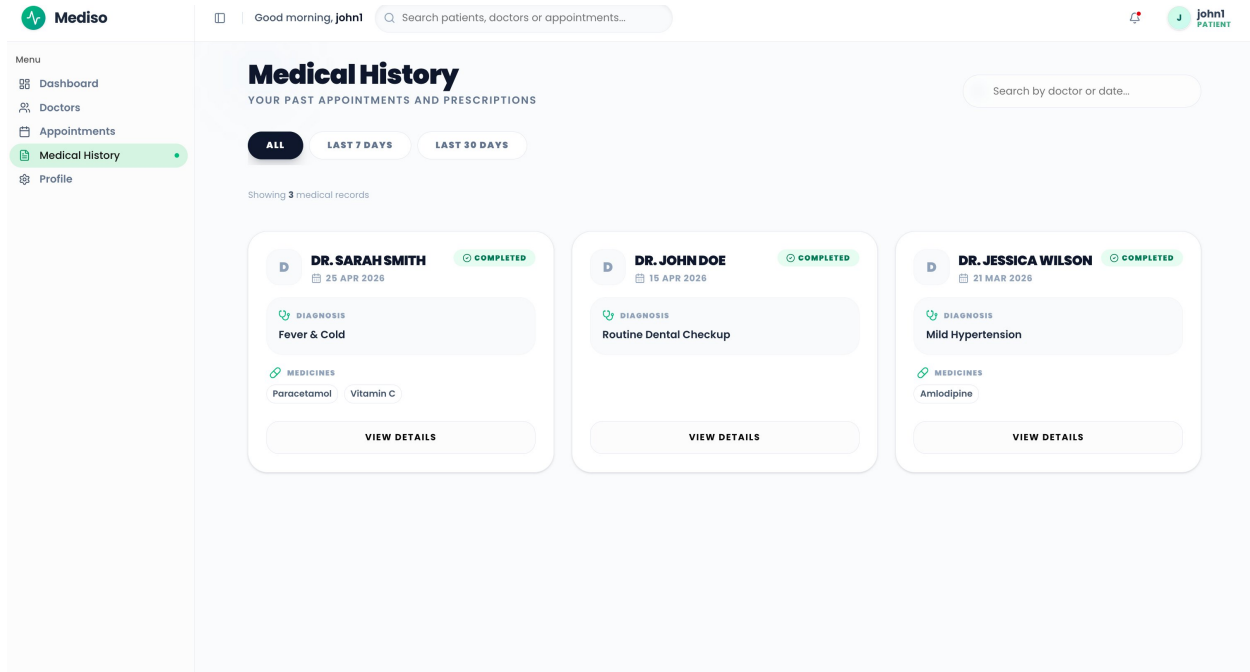


Figure 9: Medical History Page

8.6 My Profile Page

The profile page displays the patient's personal information including name, email, phone, date of birth, and blood group. An Edit Profile button opens an editable form. A Security section on the right allows the patient to update their password.

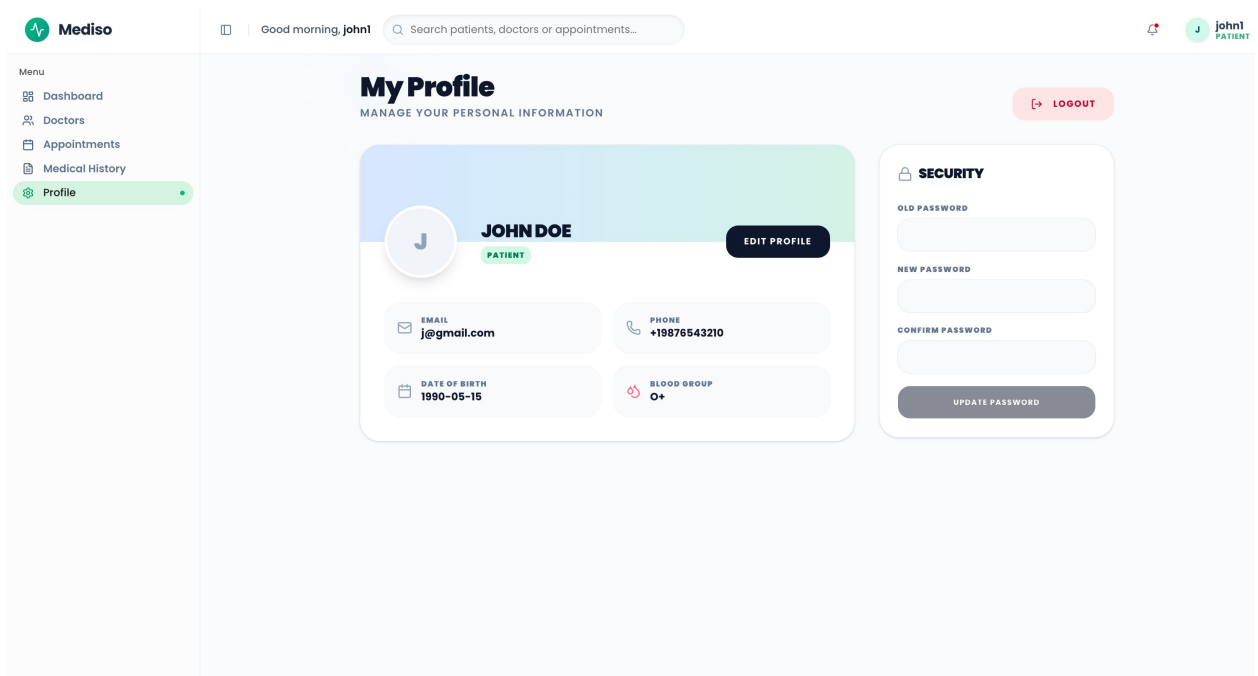


Figure 10: My Profile Page

9. Test Cases and Results

Test Case	Input	Expected Output	Result
Patient Login	Valid email and password	JWT issued, redirect to /patient/dashboard	Pass
Invalid Login	Wrong password	401 Unauthorized, error message displayed	Pass
Book Appointment	Select doctor and available slot	Appointment created, confirmation shown	Pass
Book Unavailable Slot	Select already booked slot	Error: slot unavailable	Pass
Cancel Appointment	Click Cancel on confirmed appointment	Appointment status changed to cancelled	Pass
View Medical History	Patient navigates to history page	Completed appointments with prescriptions shown	Pass
Doctor Sets Availability	Doctor adds a time slot	Slot appears in patient booking options	Pass
Doctor Adds Prescription	Doctor submits prescription form for appointment	Prescription saved, visible in patient history	Pass
Admin Manages Doctors	Admin adds or removes a doctor	Doctor list updated accordingly	Pass
Admin Manages Patients	Admin views and manages patient records	Patient records visible and editable	Pass
Role Guard	Patient tries to access /admin/dashboard	Redirected to login or patient dashboard	Pass
Token Expiry	Expired JWT sent with request	401 returned, user redirected to login	Pass
Edit Patient Profile	Patient updates phone and blood group	Profile updated, new values displayed	Pass
Search Doctors	Patient types specialization in search	Filtered doctor list shown	Pass

10. Conclusion

Mediso demonstrates how a well-structured full-stack application can meaningfully improve healthcare administration. By applying core principles of object-oriented design, the team produced a codebase that is modular, readable, and maintainable. The service layer separation, JWT-based authentication, and role-based routing together create a secure multi-tenant system where each actor operates within clearly defined boundaries.

The use of established design patterns such as Repository, Singleton, and Strategy reduced code duplication and made the system more adaptable to change. Adherence to SOLID principles ensured that each component has a clear purpose and that changes in one part of the system do not cascade unexpectedly into others.

The UML diagrams created at the outset of the project provided a clear architectural blueprint that guided development. The alignment between the class diagram, ER diagram, and the actual Prisma schema reflects disciplined design practice. The sequence diagram for appointment booking accurately describes the implemented flow, validating that the system was built according to plan.

Future improvements could include real-time notifications using WebSockets, a mobile application built with React Native sharing the same backend, integration with payment gateways for consultation fees, and enhanced analytics on the admin dashboard.